

DSperse: A Framework for Targeted Verification in Zero-Knowledge Machine Learning^{*}

Dan Ivanov¹, Spencer Graham¹, Shirin Shahabi¹, Tristan Freiberg¹, Jonathan Gold¹, and Haruna Isah¹

Inference Labs Inc.

{dan,spencer,shirin,tristan,jonathan,haruna}@inference-labs.com

Abstract. DSperse is a modular framework for distributed machine learning inference with strategic cryptographic verification. Operating within the emerging paradigm of *distributed zero-knowledge machine learning*, DSperse avoids the high cost and rigidity of full-model circuitization by enabling *targeted verification* of strategically chosen sub-computations. These verifiable segments, or “slices,” may cover part or all of the inference pipeline, with global consistency enforced through audit, replication, or economic incentives. This architecture supports a pragmatic form of *trust minimization*, localizing zero-knowledge proofs to the components where they provide the greatest value. We evaluate DSperse on three production vision models (ResNet-50, RT-DETRv2, YOLOv8) and show that spatial decomposition (autotiling) enables circuit compilation, proof generation, and verification of convolutional layers on consumer hardware. We report empirical results on memory usage, runtime, and circuit behavior under sliced and tiled configurations. By allowing proof boundaries to align flexibly with the model’s logical structure, DSperse supports scalable, targeted verification suited to diverse deployment needs.

Keywords: zero-knowledge proofs, verifiable ML, distributed inference, targeted verification

1 Introduction

As AI models are increasingly deployed in decentralized environments, ranging from distributed compute networks to on-chain inference, the need for verifiable machine learning has become more urgent. This has led to growing interest in *distributed zero-knowledge machine learning* (dzkML), which combines cryptographic proofs with modular, multi-agent inference pipelines [4].

Zero-knowledge proofs (ZKPs) offer a compelling tool for this purpose, but current approaches remain too computationally expensive to scale to real-world models [12,16]. Full-model circuitization, required for end-to-end cryptographic guarantees, introduces prohibitive cost and latency, rendering zkML infeasible for many practical deployments [17].

^{*} Project page: <https://inference-labs-inc.github.io/dspense.github.io/>

DSperse addresses this bottleneck by enabling *targeted verification*: a slice-based architecture in which only strategically selected segments of a model are circuitized and proven. These segments may include proprietary logic, safety-critical routines, or other high-value computations. Slices can be independently verified and flexibly composed, offering a modular strategy for balancing trust, performance, and deployability.

Our results suggest that distributed inference and zkML frameworks like DSperse can offer meaningful verifiability over selected subcomputations, without incurring the full cost of monolithic proofs. This makes them well-suited for deployment in real-world systems where partial verifiability offers practical benefits.

2 Related Work

The zkML landscape is comprised of verifiable training, testing, and inference. As outlined by Peng et al. [12], these collectively ensure trust in machine learning. This work focuses on inference, arguably the most exposed and latency-sensitive phase, and where ZKP-based assurances are currently most feasible [15].

A key limitation of verifiable ML inference with ZKPs is the substantial computational overhead [17]. According to Xing et al. [19], efficiency gains can be pursued through (i) tailoring proof systems to specific ML models, (ii) leveraging specialized hardware, and (iii) balancing security and privacy with efficiency. Our study adopts option (iii), which allows for selective or targeted verification.

One recent system, psvCNN [3], addresses the challenge of high proof burdens by parallelizing the circuit into independent blocks. However, psvCNN maintains an end-to-end proof model, which may still be prohibitive for frequent or large-scale inference.

DSperse takes a different approach: rather than proving the entire inference, it supports modular and selective verification of strategically chosen segments. Model slicing has been explored by Zhang et al. [20] and Zhou et al. [22], but DSperse applies this idea in a novel cryptographic setting. To our knowledge, slicing strategies have not previously been applied to zkML inference.

3 DSperse Overview

3.1 System Goals and Capabilities

DSperse is a pragmatic framework for deploying ML models in decentralized environments where full zero-knowledge inference remains prohibitively expensive. Components of a model vary in their criticality, and so too may their verification requirements. DSperse supports a risk-sensitive approach to verification, focusing resources where they matter most.

The framework provides fine-grained control over how a model is sliced into subcomputations that can be executed, circuitized, or verified independently. Model providers can selectively circuitize parts of a neural network and generate

ZKPs for those segments, while remaining parts run openly or are delegated to public infrastructure. DSperse also supports purely decentralized inference without cryptographic proof, retaining utility as a lightweight method for inference delegation with optional audit logging. The architecture exposes parameters that allow circuit designers to specify how many layers are included in each slice, enabling adaptation to resource-constrained nodes.

A model provider may choose to circuitize the entire inference as a single slice, or verify each slice independently with output-to-input consistency managed externally. This flexibility accommodates deployments ranging from partial verification of proprietary components to full-model coverage.

3.2 High-Level Architecture

DSperse provides tools for model slicing, circuit generation, and per-slice proof execution, while leaving system-level concerns (key management, consistency enforcement, result aggregation) to external infrastructure or an Orchestrator. As shown in Figure 1, the Orchestrator divides the model into sequentially dependent slices, assigns each to a Node, and coordinates intermediate values between Nodes. Each Node executes its slice, generates the witness, and produces a ZKP.

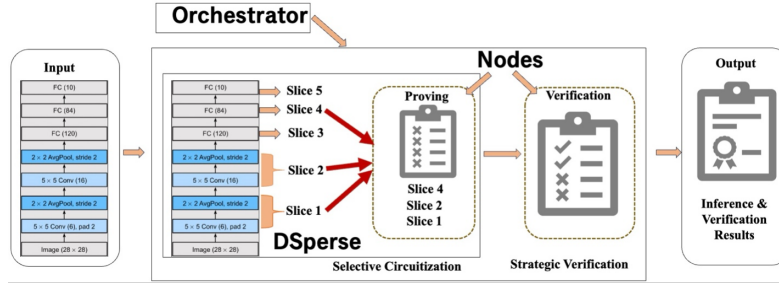


Fig. 1. DSperse framework architecture

The system does not enforce global soundness cryptographically. Correctness across slices must be ensured via additional mechanisms such as audit, redundancy, or external verification. The final output includes the model’s prediction and per-slice proofs that can be individually verified.

The flow of operations is: (1) the user submits the model and inference data; (2) the slicing module splits the model into discrete slices; (3) each prover node computes its slice’s output; (4) the proof generation module wraps the computation in a ZKP [23]; (5) the verifier node validates each proof; (6) the system may cache slice outputs or batch inputs for efficiency; (7) the output stores the prediction with per-slice proof artifacts.

3.3 Architecture Constraints

DSperse assumes a unidirectional dataflow: model inference proceeds through a fixed sequence of layers without backward connections, dynamic control flow, or reuse of intermediate tensors across segments. This structure aligns with standard feedforward networks, including convolutional nets and many transformer variants during inference. Architectures with loops (RNNs, LSTMs) are currently unsupported. Attention mechanisms that depend on activations across circuit boundaries require the entire attention block within a single slice. Parameter reuse across slices is disallowed unless it occurs entirely within one slice.

3.4 Autotiling and Spatial Decomposition

DSperse extends vertical slicing with *spatial decomposition* (autotiling) of intermediate activations. Rather than partitioning a model only along layer depth, DSperse can also decompose a single convolutional layer into smaller spatial regions that are independently executed, circuitized, and proven.

Each spatial tile is extended with an overlapping context boundary, which we term the *zone of relevancy* (ZoR), derived from the receptive field of the associated convolutional layers [10]. For a convolutional layer with kernel size k and dilation d , the effective kernel size is

$$k_{\text{eff}} = (k - 1) \cdot d + 1,$$

and the zone of relevancy (halo) on each spatial edge is

$$h = \left\lceil \frac{k_{\text{eff}}}{2} \right\rceil.$$

Given a tile budget B (the maximum number of elements per tile) and input channel count C_{in} , the maximum feasible spatial dimension for a square tile is

$$t_{\text{max}} = \left\lfloor \sqrt{\frac{B}{C_{\text{in}}}} \right\rfloor.$$

Tiling is feasible only when $t_{\text{max}} \geq t_{\text{min}}$, where $t_{\text{min}} = k_{\text{eff}} + 1$ ensures that each tile produces valid output after convolution. Each tile is padded with the halo boundary, yielding a physical tile of size $(C_{\text{in}}, t + 2h, t + 2h)$ for spatial dimension t . After independent execution, tile outputs are stitched to reconstruct the global activation tensor.

When spatial tiling is infeasible ($t_{\text{max}} < t_{\text{min}}$), the system falls back to *channel splitting*: the input tensor is partitioned along the channel dimension into $G = \lceil C_{\text{in}} / \lfloor B / (H \cdot W) \rfloor \rceil$ groups, each processed independently, with outputs summed and a bias correction applied. This ensures that every convolutional layer can be decomposed within a given memory budget, regardless of spatial constraints.

A tileable slice must contain exactly one convolutional operation as its first layer, followed only by elementwise operations (e.g., ReLU, sigmoid, multiplication). The autotiler validates that kernels are odd-sized and that padding equals the halo on all sides, ensuring that tile boundaries align with the convolution’s receptive field structure. This constraint preserves output equivalence between tiled and untiled execution.

3.5 Parallel Processing

DSperse exploits the independence of slices and tiles to parallelize proof generation across two dimensions: the *segment dimension* (vertical slicing along model depth) and the *tile dimension* (spatial tiling within a single convolutional layer). Because tiles within a layer share no data dependencies, each operates on a self-contained spatial region with its own halo, and they can be proven concurrently without synchronization.

Proof generation uses a Rayon-based work-stealing thread pool with configurable parallelism. The pool distributes tile-level and slice-level proving tasks across available cores, with no explicit barriers or locks between concurrent proof tasks. This applies uniformly to all pipeline stages: compilation, witness generation, proof generation, and verification.

A two-level caching architecture, inspired by activation checkpointing strategies [2], supports efficient parallel execution:

1. **Compiled circuit cache.** Each slice’s compiled circuit bundle is loaded once and shared across all its tiles via a mutex-protected reference. This avoids redundant compilation when the same circuit structure is reused for multiple tiles.
2. **Intermediate tensor cache.** Activation tensors produced by each slice are cached in a hash map keyed by tensor name, enabling incremental execution across the slice chain. Adjacent tiles sharing overlapping zones of relevancy benefit from boundary activation reuse: the halo region extracted during tile splitting preserves boundary data from the original input, eliminating redundant computation at tile edges.

When channel splitting is active instead of spatial tiling, the same parallel infrastructure applies: each channel group is executed independently, and outputs are accumulated via summation. The parallelism model is identical; no cross-group synchronization is required.

3.6 System Guarantees

DSperse provides a set of guarantees that reflect its pragmatic design. It offers *targeted verifiability*: only strategically selected segments of a computation are circuitized and cryptographically proven. These segments typically contain proprietary logic, sensitive parameters, or otherwise high-value components. The remainder of the inference pipeline proceeds without formal cryptographic guarantees, but may still be subject to audit, replication, or economic incentive mechanisms. This architecture enables a form of *trust minimization*. Rather than re-

quiring users to trust the entire inference process, DSperse reduces the trust surface to only those parts that lie outside the verifiable scope. Trust is not eliminated, but it is explicitly localized and, when possible, replaceable.

DSperse supports *strategic circuitization*, allowing developers to decide which subcomputations merit formal verification. Conceptually, an inference can be decomposed as:

$$z = F_{\text{pub}}^{(2)} \circ F_{\text{priv}} \circ F_{\text{pub}}^{(1)}(x),$$

where $F_{\text{pub}}^{(1)}$ and $F_{\text{pub}}^{(2)}$ are public computations (e.g., preprocessing and postprocessing), and F_{priv} is a sensitive intermediate function that is circuitized and proven in zero knowledge. DSperse certifies the correct execution of F_{priv} , without revealing its internal structure or parameters, while treating surrounding components with lighter-weight trust mechanisms. This decomposition allows teams to balance proof cost, model fidelity, and intellectual property protection.

This selective verification is embedded within a broader *pragmatic graybox architecture*. DSperse does not enforce full transparency or full secrecy. Instead, it enables mixed execution environments in which some components are openly run, others are cryptographically secured, and the rest are entrusted to context-sensitive operational or economic safeguards.

3.7 Fidelity and Model Degradation

Before participating in a ZKP, a model’s floating-point computation must be *circuit adapted*: weights and activations are quantized to fixed-point field elements, nonlinearities are replaced by surrogates, and the graph is re-expressed as arithmetic-gate constraints. We use the term *fidelity* to refer to the proximity of a circuit-adapted model’s outputs to those of the original.

Given logits $\mathbf{z}^{\text{orig}}(x)$ and $\mathbf{z}^{\text{circ}}(x)$, the *discrepancy* is:

$$D_p(x) = \|\mathbf{z}^{\text{orig}}(x) - \mathbf{z}^{\text{circ}}(x)\|_p^p = \sum_{j=1}^k |y_j^{\text{orig}} - y_j^{\text{circ}}|^p, \quad (1)$$

where $p \in \{1, 2\}$. We additionally assess softmax-level proximity. Given distributions $P = (p_1, \dots, p_k)$ and $Q = (q_1, \dots, q_k)$, the *total variation distance* is:

$$\text{TVD}(P, Q) = \frac{1}{2} \sum_{i=1}^k |p_i - q_i|, \quad (2)$$

and the *Jensen-Shannon divergence* is:

$$\text{JSD}(P, Q) = \frac{1}{2} \sum_{i=1}^k p_i \log_2 \left(\frac{2p_i}{p_i + q_i} \right) + \frac{1}{2} \sum_{i=1}^k q_i \log_2 \left(\frac{2q_i}{p_i + q_i} \right). \quad (3)$$

4 Threat and Trust Model

DSperse aims to *minimize trust* through strategic cryptographic verification, allowing developers to reduce the trust surface and isolate critical components for formal validation.

DSperse mediates between two trust perspectives. The *model provider* seeks to protect proprietary models; DSperse enables selective circuitization so sensitive components can be proven in zero-knowledge without revealing weights. The *verifier* seeks assurance that outputs result from faithful inference; when only part of the model is proven, additional mechanisms (manual validation, redundant checks, orchestration-layer trust) ensure consistency.

Each circuitized slice defines a localized *trust boundary* within which the verifier can confirm correct execution. When a slice is not circuitized, it may still be audited, or constrained by incentives or detection risk. This flexibility supports deployment models from fully auditable pipelines to economically motivated orchestration.

Each slice can be verified independently. While the inference is not yet cryptographically unified, the system supports a practical middle ground: partial verification for high-leverage subcomputations, with consistency maintained through auditability, incentives, or delegated trust. DSperse remains forward-compatible with recursive proof composition for use cases that demand maximal assurance.

5 Framework Evaluation

We evaluate DSperse on LeNet-5 [9], a compact architecture that enables controlled comparison across proving systems while isolating the effects of slicing.

5.1 Model and Slicing Strategy

Our LeNet implementation accepts $3 \times 32 \times 32$ RGB inputs and outputs 10 logits. The model is decomposed into five slices along natural architectural boundaries: two convolutional blocks (each with Conv, ReLU, MaxPool), two fully connected blocks with ReLU, and a final output layer (Figure 2). Reference implementations are available at [11,18].

5.2 Proving Systems

We benchmark using two distinct proving systems:

- **EZKL** [23]: a Halo2-based [1] system with ONNX support;
- **JSTprove** [5]: built on the Expander Compiler Collection [13], using custom arithmetic circuits [7] for GKR/sumcheck verification.

We do not attempt a head-to-head comparison; our goal is to validate that DSperse’s architecture functions across different proving stacks.

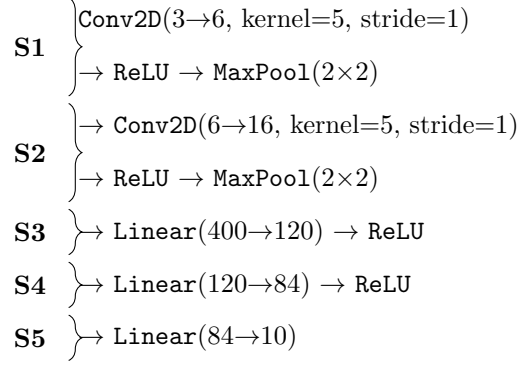


Fig. 2. LeNet-5 decomposed into five slices (S1–S5)

5.3 Evaluation Setup

We use 70 CIFAR images ($3 \times 32 \times 32$) as inputs. Memory is measured via peak RSS tracking for EZKL and internal allocation reporting for JSTprove. Timing uses wall-clock intervals for EZKL and internal instrumentation for JSTprove. Slicing is performed *before* circuit adaptation so each slice can be individually quantized.

5.4 Fidelity Results

We assess fidelity using the logit-level discrepancy D_1 , D_2 (Eq. (1)) and softmax-level TVD and JSD (Eqs. (2), (3)), averaged over 70 inputs.

For EZKL, slicing yields a modest improvement: mean D_1 decreases from 0.0076 to 0.0075 (sliced), mean D_2 from 1.01×10^{-5} to 9.93×10^{-6} . Mean TVD drops from 2.42×10^{-4} to 2.38×10^{-4} , and JSD from 1.06×10^{-7} to 1.03×10^{-7} .

JSTprove exhibits higher fidelity overall ($D_1 = 0.0012$, $D_2 = 2.71 \times 10^{-7}$) with no measurable difference between sliced and unsliced variants. Both systems preserved the predicted class identically to the original PyTorch model across all inputs.

5.5 Memory and Timing Results

For EZKL, slicing provides substantial memory benefits: peak RAM during witness generation drops from 1,049 MB to 52 MB per slice ($\sim 20\times$ reduction). Proof-stage total memory decreases from 32,263 MB to 20,106 MB (38% reduction). Figure 3 visualizes the distribution of total proof-stage memory.

For JSTprove, memory is reported as a fixed internal allocation (Table 1). Under slicing, allocated memory is approximately halved across all stages.

Table 2 presents the combined timing results for both proving systems. For EZKL, slicing reduces witness time by 77%, proof time by 66%, and verification

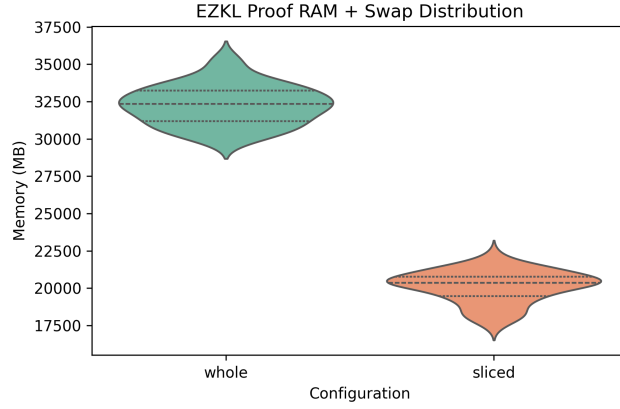


Fig. 3. EZKL proof-stage total memory: full vs. sliced

Stage	Allocated (MB)
Full Witness	160.3
Sliced Witness	80.5
Full Proof	1,085.0
Sliced Proof	544.3
Full Verify	1,082.6
Sliced Verify	543.1

Table 1. JSTprove memory allocation per stage

by 38%. JSTprove witness generation increases under slicing (0.27s to 1.52s) due to orchestration overhead in small circuits, while proof time decreases by 9%. Figure 4 shows the distribution of EZKL proof-stage wall times.

System	Stage	Mean	Std	Min	Max
EZKL	Full Witness	1.584	0.211	1.386	2.194
	Sliced Witness	0.364	0.154	0.284	0.996
	Full Proof	478.7	41.56	425.1	643.1
	Sliced Proof	163.2	11.99	151.1	215.3
	Full Verify	0.988	0.239	0.814	1.870
	Sliced Verify	0.605	0.206	0.371	0.976
JSTprove	Full Witness	0.269	0.014	0.256	0.313
	Sliced Witness	1.523	0.057	1.451	1.727
	Full Proof	9.786	0.432	9.498	11.71
	Sliced Proof	8.872	0.342	8.672	10.45
	Full Verify	9.608	0.494	9.361	12.15
	Sliced Verify	8.694	0.321	8.540	10.71

Table 2. Runtime (seconds) over 70 CIFAR inputs

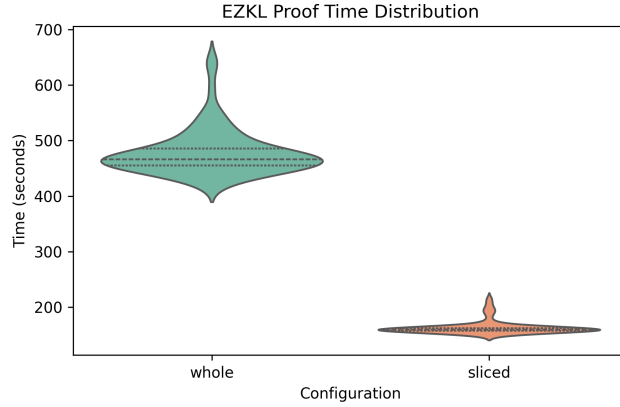


Fig. 4. EZKL proof generation time: full vs. sliced

6 Targeted Verification on Consumer Hardware

The LeNet benchmarks confirm that slicing works correctly at the mechanical level. The more pressing question is whether DSperse can bring cryptographic verification to *production-scale* vision models on consumer hardware.

6.1 Test Environment

All experiments were run on a single consumer device:

- **Hardware:** MacBook Air, Apple M3 (4P + 4E cores), 24 GB unified memory
- **Backend:** JSTprove [5] with Expander [13]
- **Timeout:** 180 s per compilation attempt

6.2 Models and Target Layers

We selected the smallest Conv layer from each model as our verification target (Table 3). These represent the least expensive convolutional operations; if these cannot be circuitized without spatial decomposition, no deeper or wider layer can be either.

6.3 Circuitization: Untiled vs. Tiled

Without spatial decomposition, none of the three layers compiled within the 180 s timeout. Autotiling (Section 3.4) at $B = 4096$ decomposes each Conv into independent tiles. A single tile circuit is compiled once and reused for every tile (Table 4).

Model	Kernel	Stride	$C_{\text{in}} \rightarrow C_{\text{out}}$	Input	Slices
ResNet-50 [6]	7×7	2	$3 \rightarrow 64$	224^2	48
RT-DETRv2 [21]	3×3	2	$3 \rightarrow 32$	640^2	478
YOLOv8 [14,8]	3×3	2	$3 \rightarrow 80$	1280^2	314

Table 3. Target Conv layers at tile budget $B = 4096$

Model	Untiled	Grid	Tile Input	Compile (s)	Constraints
ResNet-50	timeout	7×7	3×38^2	50	64,262
RT-DETRv2	timeout	20×20	3×34^2	13	32,773
YOLOv8	timeout	40×40	3×34^2	39	81,925

Table 4. Circuit compilation for the smallest Conv tile

ResNet-50 produces the largest tile because of its 7×7 kernel (3-pixel halo per side). RT-DETRv2 compiles fastest due to only 32 output channels. Compiled bundles are compact: 16 MB (ResNet-50), 3.8 MB (RT-DETRv2), 11 MB (YOLOv8).

6.4 End-to-End Targeted Verification Pipeline

Table 5 reports the full pipeline cost. Witness generation uses 8 workers; prove and verify use 4 workers, which we found optimal on the M3 (Section 6.5).

Model	Tiles	Plain (ms)	Wit (s)	Prove (s)	Vfy (s)	RAM (MB)
ResNet-50	49	31	48.4	28.6	18.4	130
RT-DETRv2	400	310	75.8	202.1	143.8	295
YOLOv8	1,600	6,840	365	1,264	992	1,316

Table 5. End-to-end verification on MacBook Air M3 (24 GB)

Table 6 isolates the per-tile cost from grid size.

Model	Tiles	Wit/tile (s)	Prove/tile (s)	Vfy/tile (s)	Total/tile (s)
ResNet-50	49	0.99	0.58	0.38	1.95
RT-DETRv2	400	1.49	0.51	0.36	2.36
YOLOv8	1,600	0.91	0.78	0.62	2.31

Table 6. Per-tile sequential cost

Per-tile costs remain stable across architectures (1.95 to 2.36 s), confirming linear scaling with tile count. ResNet-50 completes prove+verify in under 50 s for 49 tiles; RT-DETRv2’s 400 tiles finish in under 6 minutes.

6.5 Parallel Scaling

Table 7 reports prove and verify wall times for ResNet-50 as a function of worker count. Beyond 4 workers, the M3’s efficiency cores and thermal constraints reduce throughput.

Workers	Prove (s)	Speedup	Verify (s)	Speedup
1	193.0	1.0×	41.1	1.0×
2	38.2	5.1×	23.9	1.7×
4	28.6	6.7×	18.4	2.2×
8	48.8	4.0×	22.4	1.8×

Table 7. Parallel scaling for ResNet-50 (49 tiles)

The prove stage shows super-linear speedup from 1 to 2 workers (5.1×), suggesting single-threaded execution is bottlenecked by memory latency.

6.6 Circuit Characteristics

Table 8 reports Expander compiler statistics for each tile circuit. Each circuit is compiled once and shared across all tiles via a mutex-protected cache (Section 3.5).

Model	Variables	Additions	Mult.	Total Cost
ResNet-50	1,274,493	3,566,184	786,476	444,846,922
RT-DETRv2	585,007	843,879	360,492	214,346,080
YOLOv8	1,662,269	2,314,357	1,065,004	804,403,594

Table 8. Expander circuit statistics per tile

6.7 Overhead Summary

Figure 5 decomposes verification cost into total pipeline time (left) and per-tile sequential cost (right). Figure 6 presents the normalized cost profile across five pipeline metrics.

Table 9 presents per-tile time distributions. Prove times show the widest spread: YOLOv8 ranges from 2.03 s to 12.81 s per tile, reflecting scheduling variance and thermal effects. Witness generation is the most uniform stage. Verify times track prove times at a consistent discount (verify/prove ratios of 0.71 to 0.81).

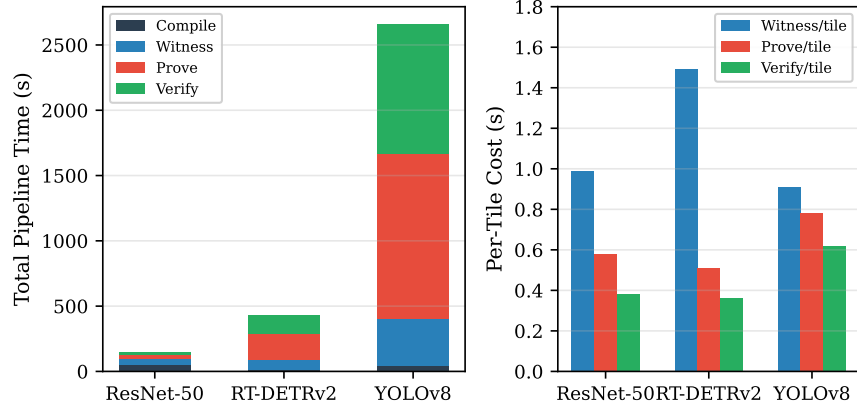


Fig. 5. Total pipeline time (left) and per-tile cost (right)

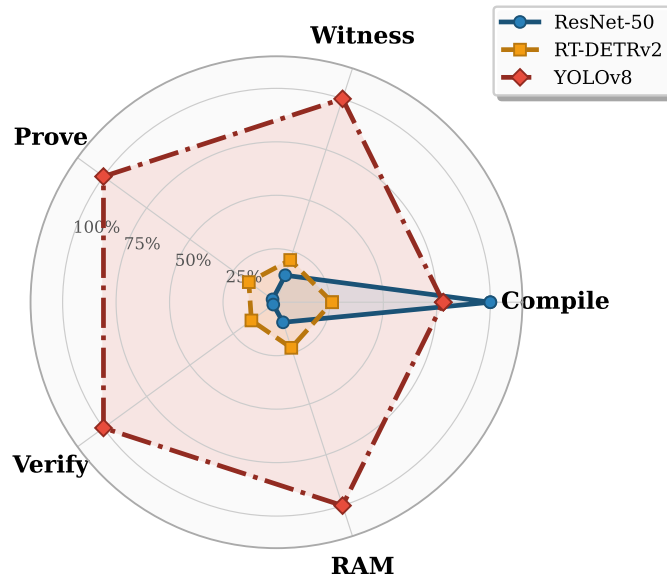


Fig. 6. Normalized verification cost across five metrics

6.8 Full-Model Verification: YOLOv8x on Server Hardware

To measure the cost of verifying *every* Conv layer, we ran YOLOv8x [8] on a Mac Studio (M3 Ultra, 24 cores, 512 GB). This model has 313 slices and 8,398 tiles (Table 10).

Model	Stage	Min (s)	Mean (s)	Max (s)
ResNet-50	Witness	0.780	0.989	1.901
	Prove	1.472	2.196	2.779
	Verify	1.310	3.551	4.384
RT-DETRv2	Witness	0.876	1.490	2.733
	Prove	2.120	4.020	7.573
	Verify	1.322	2.858	8.537
YOLOv8	Witness	0.800	1.830	3.500
	Prove	2.028	3.155	12.809
	Verify	1.474	2.475	14.271

Table 9. Per-tile time distributions (seconds)

Stage	Duration	Detail
Compile (cached)	~2.5 min	circuit reuse from prior run
Witness	~81 min	313 slices, 8,398 tiles
Prove	~36 min	104 slices, parallel 10
Verify	~31.5 min	104 slices, parallel 10
Total	~2 h 31 min	

Table 10. YOLOv8x full-model benchmark on Mac Studio M3 Ultra

All 104 Conv slices were compiled, proven, and verified. The remaining 209 slices contain operations not yet supported by JSTprove and were executed natively. This represents complete convolutional coverage.

6.9 Cryptographically Signed Real-Time Video

A custom player-detection model built on YOLOv8, with full convolutional proving, is deployed for commercial real-time inference on the SN2 Studio platform¹. Figure 7 shows a representative frame with real-time verification metrics.

Per-tile artifacts are modest: witnesses occupy 171 to 449 KB and proofs 182 to 416 KB. Effective parallelism varies with tile count: RT-DETRv2 and YOLOv8 achieve near-perfect utilization (99.5% and 99.8%), while ResNet-50 at 8 workers achieves only 49%; at 4 workers, all models achieve at least 97%.

6.10 Implications

These results establish four findings:

1. **Untiled circuitization is infeasible on consumer hardware.** Even the smallest Conv layer exceeds compile budgets without spatial decomposition.
2. **Autotiling enables consumer-device verification.** At $B = 4096$, all models compile in 13 to 50 s, with prove+verify under 50 s for ResNet-50.
3. **Parallelism should match performance cores.** Optimal throughput at 4 workers; 8 workers degrades throughput by $1.7\times$.

¹ <https://sn2-studio.inferencelabs.com/circuits/player-detection>



Fig. 7. Verified player detection on SN2 Studio

4. **Full Conv coverage is achievable on server hardware.** Every Conv layer in YOLOv8x is proven in under 2.5 hours.

7 Limitations and Future Work

Consumer-hardware scope. The consumer benchmarks verify only the smallest Conv layer per model. Extending to deeper layers requires channel splitting and more aggressive tile budgets.

Proving latency. YOLOv8’s 1,600-tile grid requires roughly 21 minutes for proving on consumer hardware. For real-time video, proof generation must be amortized across frames or offloaded to server infrastructure.

Proof composition. Each tile and slice is proven independently. DSperse does not yet support recursive proof linking; consistency between slices relies on external orchestration.

Operator support. JSTprove currently supports Conv, Gemm, and MatMul. Non-linear activations, pooling, normalization, and attention are executed natively (209 of 313 YOLOv8x slices).

Future directions. Automating slice and tile selection, extending operator support, and integrating recursive proof composition are the primary near-term goals.

8 Conclusion

DSperse introduces targeted verification as a practical approach to verifiable ML inference. Rather than requiring full-model circuitization, developers select which subcomputations to prove, at what granularity, and with which backend.

Three architectural innovations make this practical. *Vertical slicing* decomposes models along natural layer boundaries into independently provable segments. *Autotiling* with zone-of-relevancy overlap decomposes convolutional layers into spatially independent tiles small enough for consumer-device memory

budgets. *Two-dimensional parallelism* across segment and tile dimensions exploits multi-core hardware via a work-stealing thread pool with two-level caching.

The framework is *proof-system agnostic*: any proving system can be integrated without modifying the slicing or tiling logic. It is *generalizable across model families*: the same pipeline applies to LeNet-5, ResNet-50, RT-DETRv2, and YOLOv8. Autotiling and channel splitting ensure any convolutional layer can be decomposed within a given memory budget.

The combination of targeted verification and spatial decomposition enables *cryptographically signed video-vision processing*, demonstrated in commercial deployment on SN2 Studio². DSperse’s architecture is forward-compatible with recursive proof composition and automated verification-budget optimization.

9 Acknowledgments

We thank the entire Inference Labs team for their support throughout this work.

References

1. Bowe, S., Grigg, J., Hopwood, D.: Recursive proof composition without a trusted setup. Cryptology ePrint Archive, Paper 2019/1021 (2019)
2. Chen, T., Xu, B., Zhang, C., Guestrin, C.: Training deep nets with sublinear memory cost. In: arXiv preprint arXiv:1604.06174 (2016)
3. Fan, Y., Xu, B., Zhang, L., Tan, G., Yu, S., Li, K.C., Zomaya, A.: psvcn: A zero-knowledge cnn prediction integrity verification strategy. IEEE Transactions on Cloud Computing 12(2), 359–369 (2024)
4. Ghodsi, Z., Gu, T., Garg, S.: Safetynets: Verifiable execution of deep neural networks on an untrusted cloud. Advances in Neural Information Processing Systems 30 (2017)
5. Gold, J., Freiberg, T., Isah, H., Shahabi, S.: JSTprove: Pioneering verifiable AI for a trustless future. arXiv preprint arXiv:2510.21024 (2025)
6. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 770–778 (2016)
7. Inference Labs Inc.: zkml blueprints: Arithmetic circuits for neural network inference. <https://github.com/inference-labs-inc/zkml-blueprints> (2025), accessed: March 13, 2026
8. Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8 (2023), version 8.0, Ultralytics
9. Lecun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proceedings of the IEEE 86(11), 2278–2324 (1998)
10. Majurski, M., Bajcsy, P.: Exact tile-based segmentation inference for images larger than GPU memory. Journal of Research of the National Institute of Standards and Technology 126, 126009 (2021)
11. MindSpore AI: MindSpore Image Classification Models with MNIST on the Hugging Face Hub. <https://huggingface.co/mindspore-ai/LeNet> (2025), accessed: March 13, 2026

² <https://sn2-studio.inferencelabs.com/circuits/player-detection>

12. Peng, Z., Wang, T., Zhao, C., Liao, G., Lin, Z., Liu, Y., Cao, B., Shi, L., Yang, Q., Zhang, S.: A survey of zero-knowledge proof based verifiable machine learning. arXiv preprint arXiv:2502.18535 (2025)
13. Polyhedra Network: Expander Compiler Collection. <https://github.com/PolyhedraZK/ExpanderCompilerCollection> (2025), accessed: March 13, 2026
14. Redmon, J., Divvala, S., Girshick, R., Farhadi, A.: You only look once: Unified, real-time object detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 779–788 (2016)
15. Scaramuzza, F., Quattrocchi, G., Tamburri, D.A.: Engineering trustworthy machine-learning operations with zero-knowledge proofs. arXiv preprint arXiv:2505.20136 (2025)
16. Sheybani, N., Ahmed, A., Kinsy, M., Koushanfar, F.: Zero-knowledge proof frameworks: A systematic survey. arXiv preprint arXiv:2502.07063 (2025)
17. South, T., Camuto, A., Jain, S., Nguyen, S., Mahari, R., Paquin, C., Morton, J., Pentland, A.: Verifiable evaluations of machine learning models using zkSnarks. arXiv preprint arXiv:2402.02675 (2024)
18. Varshney, P.: LeNet Architecture: A Complete Guide. <https://www.kaggle.com/code/blurredmachine/lenet-architecture-a-complete-guide> (2020), accessed: March 13, 2026
19. Xing, Z., Zhang, Z., Zhang, Z., Li, Z., Li, M., Liu, J., Zhang, Z., Zhao, Y., Sun, Q., Zhu, L., et al.: Zero-knowledge proof-based verifiable decentralized machine learning in communication network: A comprehensive survey. IEEE Communications Surveys & Tutorials (2025)
20. Zhang, Z., Li, Y., Guo, Y., Chen, X., Liu, Y.: Dynamic slicing for deep neural networks. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 838–850 (2020)
21. Zhao, W., Lv, S., Lu, X., Xu, S., Wei, J., Cui, G., Du, K., Dang, Q., Liu, Y.: RT-DETRv2: Improved baseline with bag-of-freebies for real-time detection transformer. arXiv preprint arXiv:2407.17140 (2024)
22. Zhou, S., Li, T., Huang, Y., Shi, L., Wang, K., Liu, Y., Wang, H.: Neusemslice: Towards effective dnn model maintenance via neuron-level semantic slicing. ACM Transactions on Software Engineering and Methodology (2024)
23. Zkonduit Inc.: EZKL: The EZKL system. <https://docs.ezkl.xyz/> (2025), accessed: March 13, 2026